

Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	31 January 3035
Team ID	
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	4 Marks

Technical Architecture:

The Flood Prediction shall include the architectural diagram as below and the information as per the table1& table 2

Example: Rising Waters: A Machine Learning Approach to Flood Prediction

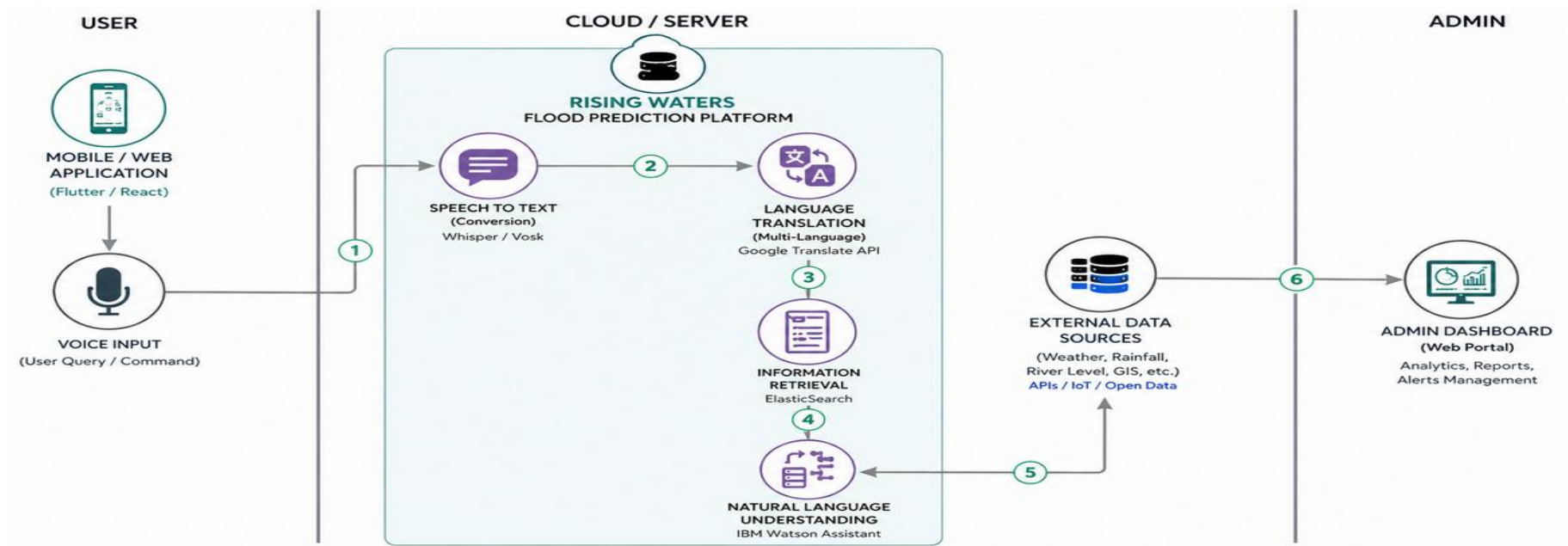


Table-1: Components & Technologies:

S. No	Component	Description	Technology
1	User Interface	Web and Mobile application for residents, farmers, disaster management authorities, and administrators to view flood predictions, alerts, reports, and maps.	React.js / Flutter, HTML5, CSS3, JavaScript, Bootstrap
2	Application Logic-1	User authentication, registration, login, user profile management, dashboard, and role-based access control.	Python (Flask/Django), REST API
3	Application Logic-2	Data collection, preprocessing, validation, feature engineering, rainfall analysis, river level monitoring, and flood risk calculation.	Python, Pandas, NumPy, Scikit-learn
4	Application Logic-3	Flood prediction engine, notification management, GIS map visualization, report generation, and emergency alert management.	Python, XGBoost/Random Forest, Leaflet.js, Firebase Cloud Messaging
5	Database	Stores user information, historical flood records, rainfall data, river water levels, prediction results, alert history, and system logs.	MySQL / PostgreSQL
6	Cloud Database	Cloud-hosted database for storing real-time weather data, historical datasets, user profiles, and prediction outputs with automatic backup and scalability.	Firebase Firestore / MongoDB Atlas / IBM Cloudant
7	File Storage	Stores machine learning datasets, CSV files, trained ML models, flood maps, reports, images, and exported PDF reports.	Local File System / AWS S3 / IBM Cloud Object Storage
8	External API-1	Retrieves real-time weather information such as rainfall, temperature, humidity, wind speed, and weather forecasts.	OpenWeatherMap API / WeatherAPI
9	External API-2	Retrieves river water level, hydrological, GIS, and satellite information for flood monitoring.	Google Maps API, NASA EarthData API, IMD API (or River Sensor API)

Table-2: Application Characteristics:

S. No	Characteristics	Description	Technology
1	Open-Source Frameworks	The application uses open-source frameworks for developing the web/mobile interface, backend services, machine learning model, data processing, and visualization dashboard.	React.js, Flutter, Python Flask/Django, Scikit-learn, TensorFlow (optional), Pandas, NumPy, Leaflet.js, Bootstrap
2	Security Implementations	Implements secure user authentication, encrypted password storage, secure API communication, role-based access control, and protection against common web attacks. Data transmission is secured using HTTPS/TLS, and application follows OWASP security best practices.	SHA-256 Password Hashing, BCrypt, JWT Authentication, HTTPS (TLS/SSL), OAuth 2.0, Role-Based Access Control (RBAC), OWASP Top 10 Guidelines, Firewall, CORS
3	Scalable Architecture	Designed using a 3-tier architecture separating presentation, business logic, and data layers. The system supports modular services, independent ML model deployment, cloud scaling, and future integration with IoT devices and additional APIs.	3-Tier Architecture, REST APIs, Docker, Kubernetes (optional), Microservices (optional), AWS/Azure/IBM Cloud
4	Availability	Ensures high availability through cloud deployment, automatic database backups, failover mechanisms, distributed cloud services, and continuous monitoring. Notification services remain available during emergencies.	AWS EC2, Azure App Service, IBM Cloud, Load Balancer, Cloud Database Replication, Firebase Cloud Messaging, Cloud Monitoring
5	Performance	Optimized for real-time flood prediction by using efficient ML algorithms, fast database queries, API caching, asynchronous processing, and lightweight frontend design. Supports concurrent users with low response time.	Redis Cache, REST APIs, Scikit-learn, NumPy, Pandas, CDN, MySQL/PostgreSQL Indexing, Async Processing, Nginx

References:

<https://c4model.com/>

<https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/>

<https://www.ibm.com/cloud/architecture>

<https://aws.amazon.com/architecture>

<https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>